

AdaptPrep: Project Explanation Guide

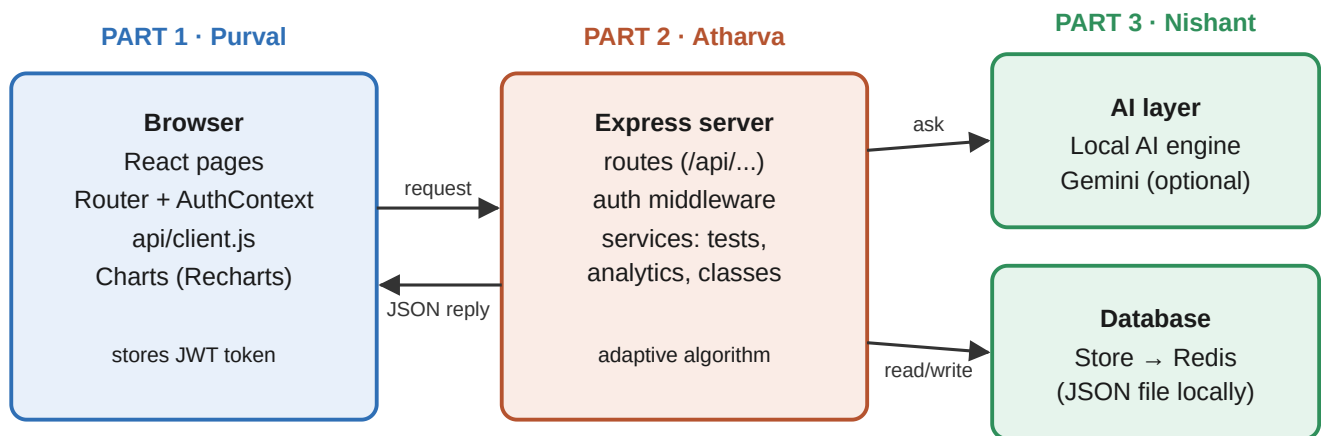
How the project is built, written and connected, in three parts for three presenters

One-line summary for the teacher: AdaptPrep is a website where JEE/NEET students take timed tests. The system remembers which topics each student is weak in and builds the next test around those topics. Students also get an AI tutor and analytics, teachers get classes and assignments, and admins manage everyone.

Who presents what

Presenter	Part	What it covers
Purval Singh 25CE1116	Part 1: Frontend (what the user sees)	React pages, routing, login state, how pages call the server, charts, design. Demo: landing page → login → dashboard.
Atharva Dorugade 25CE1124	Part 2: Backend (the brain on the server)	Express API, JWT login, roles, test creation, the adaptive algorithm, marking (+4/-1), analytics. Demo: take a test and see results.
Nishant Sunil Zemse 25CE1120	Part 3: AI, Data and Deployment (intelligence + storage + going live)	AI tutor (local engine + Gemini), database store, Redis, security, testing, Vercel deployment. Demo: AI tutor chat, admin panel.

The big picture: how the three parts connect



Every feature follows this path: page → API request → route → service → database/AI → JSON reply → page updates.

Technology used (one line each)

React 18	JavaScript library to build the UI from reusable components.
Vite	Dev server and build tool; turns our JSX into fast static files.
React Router	Changes the page without reloading (single-page app).
Node.js + Express	JavaScript on the server; Express gives us routes like <code>POST /api/auth/login</code> .
JWT + bcrypt	Login token and password hashing (passwords are never stored in plain text).
Redis (Upstash)	Cloud database used when deployed.
Vercel	Hosts the website and runs the API as a serverless function.

Part 1: Frontend (Purval Singh, 25CE1116)

"I will explain what the user sees and how the pages are built and connected to the server."

1.1 How it was made

- Created with **Vite + React**. All code is in `frontend/src/`.
- Written in **JSX**: HTML-like syntax inside JavaScript. Each screen is one component file, e.g. `pages/student/Dashboard.jsx`.
- Styling is plain CSS in `styles/app.css`, with a light and dark theme.
- Icons come from **lucide-react**, and charts come from **Recharts**.

1.2 Folder structure (what is where)

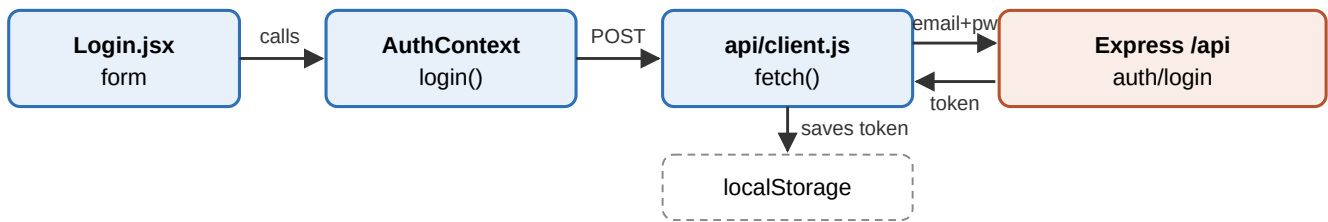
Folder	Contents
<code>pages/public</code>	Landing, Login, Signup, NotFound
<code>pages/student</code>	Dashboard, Practice, TestRunner (the exam screen), Results, History, Analytics, StudyPlan, Revision
<code>pages/teacher</code>	Dashboard, Assignments (list / new / detail)
<code>pages/admin</code>	Overview, Users, Classes, Audit log, System
<code>pages/shared</code>	Tutor, Classes, Announcements, Settings, QuestionBank (used by more than one role)
<code>components/</code>	Reusable pieces: buttons, cards, charts, chat panel, question editor
<code>context/</code>	AuthContext (who is logged in) and UiContext (theme)
<code>api/client.js</code>	The single place that talks to the backend

1.3 Routing: one website, many pages

`App.jsx` lists every URL and which component shows it. Pages are **lazy-loaded**, so a page's code downloads only when it is opened, which makes the first load faster. After login, every page sits inside `AppShell` (sidebar + top bar), and the sidebar changes according to the user's role.

```
const Practice = lazy(() => import('./pages/student/Practice'));
<Route path="/login" element={<Login />} />
<Route path="/app" element={<AppShell />>
  <Route path="practice" element={<Practice />} /> // → /app/practice
</Route>
```

1.4 How the frontend talks to the backend (integration)



Login flow on the frontend. After this, every request automatically carries the token in the "Authorization: Bearer ..." header.

- **api/client.js** wraps `fetch()`: it adds the base URL `/api`, attaches the token, converts the reply to JSON and turns errors into a readable message.
- If the server answers **401** (token expired), the client calls a handler that logs the user out and shows "Your session has ended".
- **AuthContext** is React Context: it holds `user` globally so any page can know the user's name and role.
- In development, Vite **proxies** `/api` to port 5000, so frontend and backend look like one website.

What to say: "The frontend is a React single-page application. Each screen is a component, React Router switches between them without reloading, and all server communication goes through one file, `api/client.js`, which attaches the login token automatically. The user's login state is shared using React Context."

1.5 Key screens to demo

Screen	What to point out
Landing page	Public page with features and a call to sign up.
Login / Signup	Form validation; demo accounts for each role.
Student Dashboard	Readiness ring, stat cards, score-trend line chart, accuracy-by-subject bar chart (Recharts).
Practice → TestRunner	Timer, question palette, mark-for-review, auto-save of answers.
Results	Score, correct/incorrect/skipped, explanation for every question.
Study Plan	Weekly plan; ticked items saved in the browser (localStorage).
Dark mode	Toggle in the top bar (UiContext).

1.6 React concepts used (be ready to name them)

- **Components and props:** e.g. a `Stat` card receives a title and value.
- **useState:** local data such as the selected answer and timer.
- **useEffect:** load data when a page opens; run the countdown timer.
- **Context API:** global login user and theme.
- **Custom hooks:** `useApi()` loads data and handles loading/error states; `useStartTest()` starts a test and navigates to the runner.
- **Lazy loading + Suspense:** shows a loader while a page's code downloads.

```
// lib/useApi.js: every page loads data the same way
const analytics = useApi('/analytics/me'); // loads data, tracks loading + errors
...
<TrendChart data={...} /> // Recharts line chart
```

Possible teacher questions (Part 1)

Q: Why React and not plain HTML? We have about 30 screens that share parts (sidebar, cards, charts). Components let us write each part once and reuse it.

Q: What is a single-page application? The browser loads one HTML page; JavaScript swaps the content as you navigate, so it feels instant.

Q: Where is the token stored, and is that safe? In localStorage. It expires after 7 days and the server can revoke it (for example after a password change).

Q: What does Vite do? It runs the development server with instant reload, and `npm run build` bundles and minifies everything into the `build/` folder.

Part 2: Backend (Atharva Dorugade, 25CE1124)

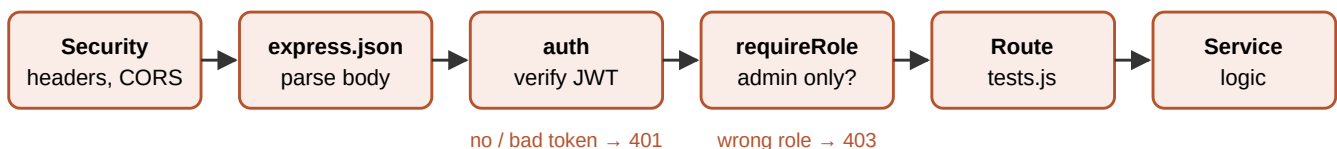
"I will explain the server: how requests are handled, how login works, and how adaptive tests are built and marked."

2.1 How it was made

- **Node.js + Express**, all code in `backend/`. `app.js` builds the app; `server.js` starts it on port 5000.
- Organised in layers: **routes** (URLs) → **middleware** (checks) → **services** (logic) → **store** (data).

Folder	Job
<code>routes/</code>	auth, me, tests, analytics, classes, assignments, questions, ai, announcements, admin
<code>middleware/</code>	auth (JWT check), security (headers, rate limit, errors), sync (database)
<code>services/</code>	attempts (tests + marking), analytics, questions, users, access, audit
<code>data/</code>	Question bank: 600 JEE/NEET questions with explanations

2.2 Life of a request



Each request passes these checks in order; if a check fails, the request stops there with an error.

```
app.use('/api/auth', require('./routes/auth')); // public: login, signup
app.use('/api/tests', auth, require('./routes/tests')); // must be logged in
app.use('/api/admin', auth, requireRole('admin'), require('./routes/admin')); // admins only
```

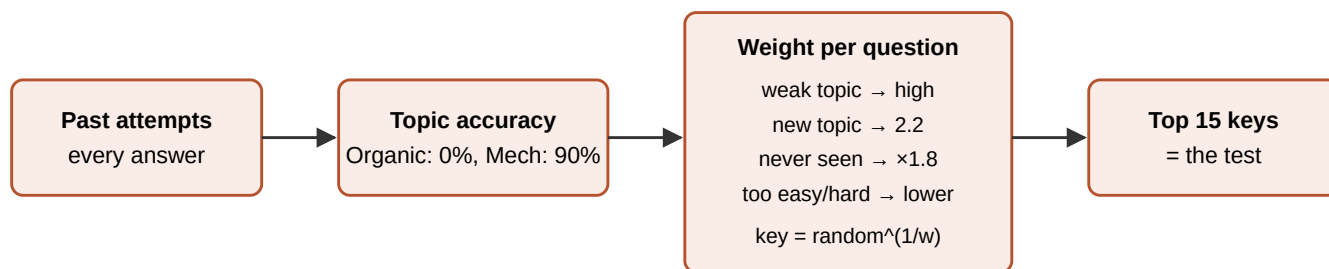
2.3 Login and security (JWT)

1. Signup: the password is hashed with **bcrypt** and only the hash is saved.
2. Login: `bcrypt.compare` checks the password, then the server signs a **JWT** containing the user id, role and a "token version".
3. Every later request: the `auth` middleware verifies the signature and loads the user. If the account is suspended, or the password or role changed (token version increased), the old token stops working immediately.
4. A **rate limiter** blocks repeated wrong-password attempts.

What to say: "The backend is a REST API in Express. Each URL goes through middleware that checks the JWT token and the user's role, then a service does the actual work. Passwords are hashed with bcrypt, and tokens can be revoked using a token version number."

2.4 The main idea: adaptive tests

When a student clicks "**Start AI Adaptive test**", the server picks 15 questions using the student's history:



Weighted random sampling: weak topics appear more often, but every test is still different.

```
let weight = acc === null ? 2.2 : 1 + (1 - acc) * 3;    // weaker topic → bigger weight
if (!seen.has(q.id)) weight *= 1.8;                  // prefer new questions
if (acc > 0.75 && q.difficulty === 'easy') weight *= 0.4; // don't waste time on easy ones
if (acc < 0.4 && q.difficulty === 'hard') weight *= 0.5; // don't overwhelm
return { q, key: Math.random() ** (1 / weight) };    // pick the highest keys
```

Simple example: if the student's Organic Chemistry accuracy is 0%, its weight is $1 + 3 = 4$, and $4 \times 1.8 = 7.2$ for unseen questions. Mechanics at 90% gets 1.3. So Organic questions are about five times as likely to appear.

2.5 Taking and marking a test

1. `POST /api/tests/start` creates an **attempt** with the question list and a **deadline** (the timer is on the server, so it can't be cheated).
2. While the student answers, `PATCH /api/tests/attempts/:id` saves progress, so a refresh doesn't lose answers.
3. `POST /api/tests/attempts/:id/submit` (or time up → auto-submit): marks = **$4 \times \text{correct} - 1 \times \text{wrong}$** . Correct answers are sent to the browser only *after* submission.
4. **Analytics service:** combines all attempts into accuracy per subject/topic, streaks, weak/strong topics and **Readiness = 60% accuracy + 40% syllabus coverage**.

2.6 Teacher and admin features

Teachers: create classes (students join with a code), write questions, publish assignments with due dates, view each student's analytics. Admins: suspend users, change roles, view all classes, and see the **audit log**, which records every important action.

What to say: "The adaptive algorithm gives every question a weight based on how weak the student is in that topic, then uses weighted random sampling to pick 15. The test timer runs on the server, and marking follows the real JEE pattern of +4 and -1."

Possible teacher questions (Part 2)

Q: What is REST? Each resource has a URL, and the HTTP verb says the action: GET reads, POST creates, PATCH/PUT update, DELETE removes. Data is sent as JSON.

Q: What is middleware? A function that runs before the route, e.g. checking the token. It either calls `next()` or stops with an error.

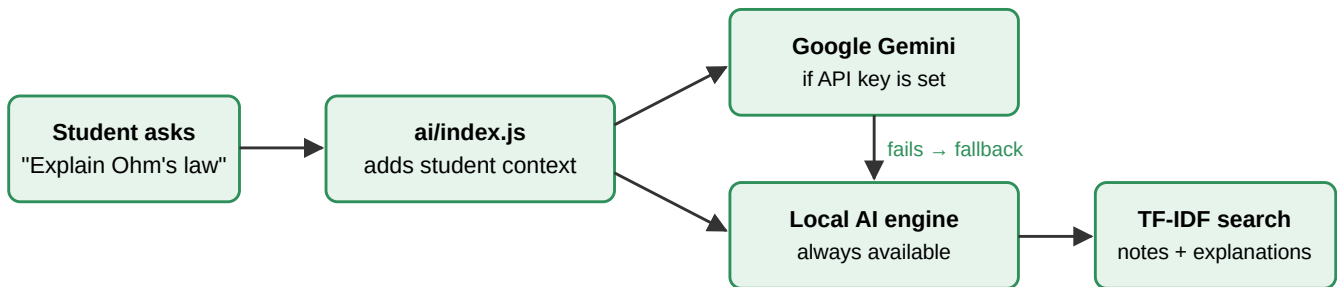
Q: Why not just pick the 15 weakest questions? Tests would repeat. Randomness weighted by weakness gives variety while still focusing on weak areas.

Q: Can a student see answers early? No. The server removes correct answers from the response until the attempt is submitted.

Part 3: AI, Data and Deployment (Nishant Sunil Zemse, 25CE1120)

"I will explain the AI tutor, where the data is stored, how we keep it secure and tested, and how the site is put online."

3.1 The AI layer: two engines, never breaks



The orchestrator tries Gemini only if a key is configured; any error falls back to the local engine, so AI features always work.

- **Context:** before answering, the AI is told the student's accuracy, weak topics and streak, so answers are personal ("You are weak in Organic, let's practise...").
- **Local engine** (`localEngine.js` + `retrieval.js`): uses **TF-IDF**, a classic search technique that scores which notes share the most important words with the question, then builds the answer from the best match. It needs no internet and costs nothing.
- **AI question generator** (`generator.js`): makes new numeric questions (e.g. kinetic energy), computes the correct answer, and builds wrong options from common student mistakes.
- **Gemini** (`gemini.js`): called with `fetch` through Google's REST API when `GEMINI_API_KEY` is set.

3.2 Where data is stored

All data (users, questions, attempts, classes, assignments, announcements, chats, audit) goes through one module, `db/store.js`, with simple functions: `find`, `insert`, `update`, `remove`.

Where it runs	Storage
On a laptop	Kept in memory and saved to a JSON file (<code>storage/db.json</code>) with safe atomic writes.
Online (Vercel)	Upstash Redis cloud database. Before each request the latest data is loaded (only if it changed); after the request only the changed records are saved.

Because every part of the code uses the same store functions, we switched storage from a file to Redis without changing any route code. This is the **separation of concerns** principle.

What to say: "The AI has two engines: Gemini when a key is available, and our own local engine, which uses TF-IDF search over study notes. If Gemini fails, we fall back automatically. Data goes through one store module, so we could move from a JSON file to a Redis cloud database without touching the rest of the code."

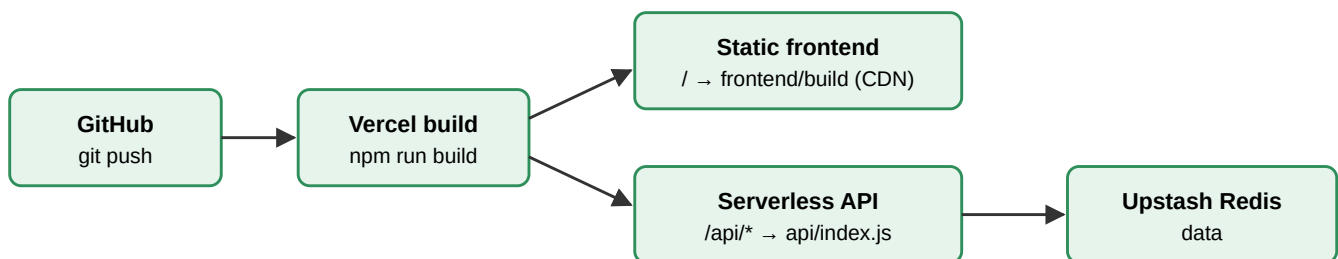
3.3 Security checklist

Threat	How we handle it
Stolen password database	bcrypt hashes; plain passwords are never stored
Fake / expired login	JWT signed with a secret; verified on every request
Student opening admin pages	Role check on the server (<code>requireRole</code>), not just hidden buttons
Password guessing	Rate limiting on login
Cheating on tests	Server-side timer; answers hidden until submit
Bad input	Every field validated (email format, password length, allowed values)
Who did what?	Audit log of important actions

3.4 Testing

The project has **17 automated tests** (`npm test` , Node's built-in test runner). They start the real server with a temporary in-memory database and call the API like a browser would: signup validation, login, wrong passwords, role blocking, taking and marking a test, token revocation, and saving to the database. All 17 pass.

3.5 Deployment: how the site goes online



Every push to the main branch rebuilds the site automatically. vercel.json tells Vercel what to build and how to route /api.

```
// vercel.json (simplified)
"buildCommand": "npm --prefix frontend run build",
"outputDirectory": "frontend/build",
"rewrites": [
  { "source": "/api/(.*)", "destination": "/api/index" }, // backend
  { "source": "/(.*)", "destination": "/index.html" } // React pages
]
```

Serverless means Vercel starts our Express app only when a request arrives; we don't manage any server. That is why the data must live in an external database (Redis): the function's own disk is temporary.

What to say: "We deploy on Vercel. The React build is served from a CDN, and the Express API runs as a serverless function. Because serverless disks are temporary, data is stored in Upstash Redis. Seventeen automated tests check the main flows."

Possible teacher questions (Part 3)

Q: Is this real AI? Both kinds: Gemini is a large language model, and the local engine is classic information retrieval (TF-IDF) with templates. The local one keeps the app free and offline-capable.

Q: Why Redis instead of MySQL? It's free on Vercel with one click, needs no schema setup, and our data is stored as JSON documents. The store design means we could move to PostgreSQL later.

Q: What happens if two users save at the same time? Each save is an atomic transaction that increases a version number. If another server changed the data in between, ours reloads before the next request.

Closing line (any presenter)

"In short, React shows the pages, Express makes the decisions, the adaptive algorithm personalises every test, the AI explains concepts, and Redis plus Vercel keep it running online. Future work: IRT-based ability scoring, a mobile app and more question types."